API preparation

Table of Contents

1.1 API Authentication

1.2 Common API Conventions

1.3 Status Codes

1.4 Human Handoff Webhook Events

1.5 Client Tools

1.6 Message Webhook

1.7 Detect New Messages

1.8 Parser Auto-Detection Results

API Authentication

MaiAgent uses API keys for authentication.

Authentication Method

All API requests must include an API key in the HTTP Header in the following format:

```
const headers = {
  "Authorization": "Api-Key YOUR_API_KEY_HERE",
  "Content-Type": "application/json"
}
```

The header prefix is `Api-Key`, not `Bearer`. For rules on which member a key is bound to, how permissions are determined, and when a key becomes invalid, see Common API Conventions: Authentication.

How to Obtain an API Key

There are two ways to obtain an API key. Both types of keys are used in the same way:

Self-issued by a member: Any active organization member can create a key for themselves in the admin console by selecting the avatar in the upper-right corner → Account → API Keys. No additional permissions are required.

Issued by an administrator: Administrators with the "API Key Management" permission can issue keys for specific members under Organization Settings → API Key Management, and configure an expiration time and requests per minute (RPM).

For instructions, see API Key Management in the user manual: Creating Your Own Key and Creating an API Key as an Administrator.

> Security reminder: Keep your API key secure and do not share it publicly Key format: An API key consists of two alphanumeric parts, a prefix and a secret, separated by .. Include the entire key when making requests

> The full key is displayed only once, when it is successfully created. The list subsequently shows only some characters, and the full key cannot be viewed or recovered. If you did not save it, create a new key, update the services that use it, and then revoke the old key.

Cannot Find an Old System-Issued Key

The API Keys page in your personal account lists only keys you created yourself. When you first join an organization (including your first login through AD), the system may automatically issue a key with no expiration date, but it does not provide the full value, so you cannot obtain that key to make API calls. To call the API, create a new key on your personal page and save the full value immediately after creation.

For keys issued automatically by the system or on behalf of a member by an administrator, only members with organization key management permissions can view their owner, prefix, issuance source, status, and expiration date under Organization Key Management. This page does not display the full value. If you do not have this permission, ask the organization owner or a member with organization key management permissions to help identify or revoke the old key.

API Key Permissions

When you call the API with an API key, your permissions match the key owner's role permissions in that organization. The API returns 403 Forbidden if the required permissions are missing.

Write operations on members (`members`) and roles (`groups`) require the Organization Overview subpermission (under Organization Permissions). Organization owners are exempt from this requirement. This applies to the following operations:

Add a member: `POST /api/v1/organizations/{organizationPk}/members/`

Remove a member: `DELETE /api/v1/organizations/{organizationPk}/members/{id}`

Change a member's roles: `POST /api/v1/organizations/{organizationPk}/members/{id}/update-groups/`

Create, update, or delete roles: `POST`
`PUT`
`PATCH`
`DELETE /api/v1/organizations/{organizationPk}/groups/…`

Add or remove role members in bulk, or assign or remove a role's AI assistants, inboxes, knowledge bases, and databases: `/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-*…`

Other write operations on members and roles (such as importing members in bulk) are also subject to this requirement

> Permissions are evaluated at the subpermission level. If the parent Organization Permissions checkbox is unchecked in the role settings, this does not mean the child Organization Overview checkbox is unchecked. To troubleshoot a 403 response, go to Organization Settings → Roles in the admin console, expand Organization Permissions, and check whether Organization Overview is selected for the key owner's role.

Error Handling

When authentication fails, the API returns the HTTP error code 401 Unauthorized:

Missing, malformed, or invalid API key

A missing key, an incorrectly formatted header, or an invalid key (nonexistent, expired, disabled, revoked, or bound to a member who has been disabled or removed from the organization) all produce the same response, without specifying the reason:

```
{
  "detail": "Authentication credentials were not provided."
}
```

Troubleshooting: Check that the request header is `Authorization: Api-Key YOUR_API_KEY` and that the API key is complete, with no extra spaces or characters. If authentication still fails with the correct format, check in API Key Management that the key is not disabled, expired, or revoked, and that its associated member is still active.

Using an API key after `Bearer`

The platform validates it as a login token and returns a different response:

```
{
  "detail": "Given token not valid for any token type",
  "code": "token_not_valid",
  "messages": [ … ]
}
```

Troubleshooting: Change the prefix to `Api-Key`.

`401` responses include a `WWW-Authenticate` header, whose value is `Bearer realm="api"` for most endpoints. Use the status code to identify the error; do not use this header to decide which type of credential to send. See Common API Conventions: Authentication Error Responses. A valid key with insufficient permissions results in `403`; see API Key Permissions above.

Common API Conventions

This page covers the rules shared by all platform APIs: URLs, authentication, rate limits, error responses, pagination, field formats, and upload limits. See “API Endpoint Documentation” in the left sidebar for each endpoint's fields and examples. These rules apply to all endpoints and are not repeated on individual endpoint pages. Read this page before you start coding.

> For UI instructions on creating and managing API Keys, see API Key Management in the user manual. To view call logs, see API Call Logs. For integration steps from scratch, see Quickstart in the technical manual.

Base URL and Version Prefix

The SaaS API URL is https://api.maiagent.ai/api/v1/. For on-premises and private cloud deployments, replace the domain with your company's platform domain.

`/api/` and `/api/v1/` expose the same endpoints, with identical paths, fields, and permissions.
AI Gateway (OpenAI-compatible endpoints) and SCIM are available **only under** `/api/v1/`.
Always use `/api/v1/` for new integrations.
Endpoint paths end with
. Omitting the trailing
returns a `301` redirect to the URL with
appended. Some HTTP clients change `POST` to `GET` when following redirects, so call the URL with the trailing
directly.

Authentication

For server-to-server calls, pass an API Key in the `Authorization` header:

```
Authorization: Api-Key
```

A key consists of an alphanumeric prefix and secret, separated by `.`. Include the entire key.

Rule	Description
Each key is bound to one member	Calls use the organization that member belongs to; requests do not need a separate organization ID

Rule	Description
Permissions match the member's roles	There are no endpoint-specific scopes. The key can perform any action the member can. Insufficient permissions return `403`; see API Key Permissions
Disabling the member invalidates the key	The key becomes unusable immediately when the member is disabled or removed from the organization
Expired, disabled, or revoked keys	Return `401`, just like a nonexistent key; the response does not disclose the reason
The full key is shown only once	Only part of the key is shown after creation. Create a new key if you lose it

For where to create and manage keys, see API Authentication.

Authentication Failure Responses

A missing key, an incorrectly formatted header, or an invalid key (nonexistent, expired, disabled, revoked, or bound to a member who has been disabled or removed from the organization) all return `401` with the same content:

```
{"detail": "Authentication credentials were not provided."}
```

An invalid key also produces the message that no authentication credentials were provided. The platform treats an unrecognized key as missing credentials and does not disclose the specific reason.

The response includes a `WWW-Authenticate` header. For most endpoints, its value is `Bearer realm="api"` (these endpoints also accept OAuth access tokens). For endpoints that accept only API Keys, such as `api-key-context` below, the value is `Api-Key`. Use the `401` status code to detect authentication failures; do not use this header's value to decide which credentials to send.

If you put an API Key after `Bearer`, the platform validates it as a login token and returns `401` with different content:

```
{"detail": "Given token not valid for any token type", "code": "token_not_valid",
"messages": [ … ]}
```

If you receive this response, change the header to `Api-Key`.

> Recommended setup for integration keys: Create a dedicated integration member, assign only the roles it needs, then issue a key for that member and set an expiration date. A key bound to a personal account becomes invalid when the employee leaves and their account is disabled, interrupting the integration. Keep API Keys on the server only: the platform API allows cross-origin (CORS) calls only from approved domains, and keys should never be exposed in the browser.

Identify the Organization Associated with a Key

To check which organization a key belongs to at your own gateway, call `GET /api/v1/users/api-key-context/`. The response is `{"organizationId": "…"}`. This endpoint accepts only API Keys and does not count toward the key's RPM.

Other Credentials

To connect an MCP client as a member, see MaiAgent as an OAuth Authorization Server in the technical manual. For member tokens used to embed MaiGPT in your own system, see Integrating MaiGPT: Member Identity Embedding.

Rate Limits

The platform API has **no global default limit**. Only the following limits apply:

Scope	Limit	Counting Method	Who Can Adjust It
Requests per API Key (RPM)	Set by an administrator; unlimited if not set	Per key, per 60 seconds	The organization's API Key Management
AI Gateway tokens (TPM)	Set by an administrator; unlimited if not set	Per key, per minute; Gateway inference only	Same as above; see AI Gateway Limits
SCIM endpoints	100 requests/minute	Per organization	Fixed
`web-chats/{id}/setup-contact-credentials/`	30 requests/minute	Per source IP	Fixed
Pricing queries that require no login (`credit/items/`, `credit/pricing-rules/`)	60 requests/minute	Per source IP	Fixed
`auth/login/`	10 requests/minute by default	Per source IP	Platform configuration

Scope	Limit	Counting Method	Who Can Adjust It
Web Chat requests without credentials (continuing conversations, uploading attachments, etc.)	10–30 requests/minute, depending on the endpoint	Per source IP	Fixed

A key's RPM applies to all platform APIs it can call, not just AI Gateway. If an endpoint also has its own limit, such as a fixed limit in the table, API Key calls are subject to both limits. Each is counted separately; whichever is reached first returns `429`, and `Retry-After` uses the longer required wait. There are two exceptions: the endpoint for identifying the organization associated with a key does not count toward the key's RPM; pricing queries that require no login (`credit/items/`, `credit/pricing-rules/`) do not recognize credentials, so even calls with an API Key are subject only to the per-source-IP limit and do not count toward the key's RPM.

Requests using OAuth access tokens or back-office login sessions are not subject to key RPM limits.

Exceeding a limit returns `429` with a `Retry-After` header in seconds. Wait for that duration before retrying; do not resend immediately.

```
{"detail": "Request was throttled. Expected available in 42 seconds."}
```

Error Responses

The platform API has no unified error code table. Handle errors based on the **HTTP status code**, then use `code` for more specific handling when present. **Do not match the text in `detail`**: it is translated according to `Accept-Language`, and its wording may change.

Response Shapes

Shape	Example	Common Uses
`detail` object	`{"detail": "You do not have permission to perform this action."}`	Authentication, permissions, not found, rate limits, plans, and Credits; a few responses also include `code` or other fields
Field object	`{"name": ["This field is required."]}`	Field validation failures; keys are camelCase field names, and errors not tied to a single field appear in `nonFieldErrors`
String array	`["…"]`	A few validation errors without an associated field

Exceptions:

AI Gateway OpenAI-compatible endpoints use an OpenAI-format `error` object; see AI Gateway Error Format.

SCIM endpoints return request body errors, not-found errors, and conflicts according to the SCIM 2.0 specification (`schemas`, `status`, `detail`).

Nonexistent URLs (incorrect paths or missing resource IDs) return `404` with HTML content, not JSON. `5xx` responses are not guaranteed to be JSON either. Check `Content-Type` before parsing a response.

Status Codes

Status Code	Meaning	Recommended Handling
`200` `201` `204`	Success; `204` has no response body and is common for deletions	—
`202`	Accepted but not yet complete; for example, attachments are still being indexed during a synchronous reply, returning `status: "pending_attachments"`	Retry later; see Cases That Do Not Enter Streaming
`301`	The path is missing a trailing	Use the URL with
`400`	Field validation failed; most endpoints return `{"detail": "Invalid UUID format."}` when a resource ID is not a valid UUID	Correct the request; do not retry
`401`	Credentials are missing or invalid; see Authentication Failure Responses	Check the key and header format; do not retry
`402`	Insufficient plan coverage, Credits, or quota; see below	Notify the organization administrator
`403`	Insufficient role permissions, the feature is not enabled for the organization, the organization is disabled, or usage has reached the limit on a character-based billing plan	Adjust role permissions or contact your representative
`404`	The resource does not exist, belongs to a different organization than the key, or the URL is incorrect	Check the ID, the key's organization, and the path
`409`	Conflicts with the resource's current state; for example, the same job is already running or a resource to be deleted is still referenced	Retrieve the latest state before deciding what to do; do not resend the same request unchanged
`429`	Rate limit exceeded	Back off according to `Retry-After`
`5xx`	Platform or upstream service error	Retry with exponential backoff

A `409` response is a `detail` object. Some endpoints include fields needed to handle the conflict, such as when an FAQ import is already running for a knowledge base:

```
{"detail": "An FAQ import is already running for this knowledge base.", "jobId": "…"}
```

402: Insufficient Plan Coverage, Credits, or Quota

Condition	Response Content	Includes `code`?
The organization has no active paid plan	`{"detail": "Your organization has no active paid plan. …", "code": "no_active_plan"}`	Yes, `no_active_plan`
The organization has insufficient Credits	`{"detail": "Insufficient credits …"}`; wording varies by endpoint	No
A member's personal quota is exhausted or unallocated, or their role's quota is exhausted	`{"detail": "Personal credit quota exhausted. …"}`; wording varies by situation	No

Only the absence of an active paid plan includes `code`. Responses for insufficient Credits or exhausted quotas contain only `detail`. Currently, the response alone cannot distinguish insufficient organization Credits from an exhausted personal or role quota. Ask the organization administrator to check in the back office.

Credits and quotas are checked on write requests that incur usage, such as sending messages, uploading and searching knowledge base files, and crawling web pages. Read-only `GET` requests do not return `402` for these reasons.

AI Gateway OpenAI-compatible endpoints return the OpenAI format: `{"error": {"type": "billing_error", "code": "insufficient_credits", …}}`. The same `code` is used when there is no active plan.

These `402` responses do not occur when Credits billing is disabled on the platform, as in some on-premises deployments.

Pagination

Most list endpoints use page-number pagination:

Parameter	Description
`page`	Page number, starting at 1; out-of-range values return `404` with `{"detail": "Invalid page."}`
`page_size` (also accepts `pageSize`)	Items per page; defaults to 20, with a maximum of 100. Values above 100 are capped at 100 without an error

The response format is `{"count": …, "next": "…", "previous": "…", "results": [ … ]}`. Use the `next` URL directly to retrieve the next page; it is `null` when no next page exists.

Two other cases:

Cursor pagination: The conversation list (`GET /api/v1/conversations/`) and message list (`GET /api/v1/messages/`) use the `cursor` parameter to paginate from newest to oldest. Use `next` to retrieve older data. The message list response includes `count`; the conversation list does not. Within each page of the message list, messages are ordered from oldest to newest.

No pagination: Some lists return an array directly, such as `GET /api/v1/credit/items/`.

Refer to each endpoint's response examples to determine which type it uses.

Field Formats

Naming: Response keys always use camelCase. Request JSON and query parameters accept both camelCase and snake_case. Some free-form field contents are preserved without case conversion, such as JSON Schema, custom HTTP headers, and metadata.

Time: Most time fields return Unix timestamps as millisecond strings, such as `"1759536000000"`, with second-level precision (the last three digits are always `000`). A few fields use ISO 8601 strings; refer to the endpoint page.

Decimals: Decimal fields such as amounts and unit prices are returned as strings, such as `"316.8000000000"`. Use a decimal type to avoid floating-point errors.

IDs: Resource IDs are usually UUIDs.

Language: Error messages are translated according to the `Accept-Language` header (for example, `ja`). Messages without a translation remain in English. Do not rely on message text.

Upload Size Limits

Upload limits are configured by the platform and may change or differ for on-premises deployments. Use the response from `GET /api/v1/allowed-file-specs/` as the authority. This endpoint requires no authentication and lists allowed file extensions and `maxSize` (in bytes) for each purpose. The production values as of October 2026 are:

Purpose	Limit
Knowledge base files	100 MB
Conversation attachments: documents, spreadsheets, presentations, and audio files	10 MB
Conversation attachments: images	5 MB
Meeting recording audio files	100 MB

For presigned uploads, `POST /api/v1/upload-presigned-url/` checks `fileSize` and the extension when issuing the upload URL. Exceeding the limit or using a disallowed type returns `400`. For the workflow, see Presigned File Upload Mode in the technical manual.

Excel imports for knowledge base metadata, tags, and FAQs share the knowledge base file limit.

Frequently Asked Questions

- ▶ Why do I keep getting 401 even though my key is correct?
- ▶ Can I restrict a key to specific APIs?
- ▶ Why do examples starting with `/api/` work as well as `/api/v1/`?

Status Codes

When sending HTTP requests through the API, use the following status codes to determine whether a request succeeded:

Status code descriptions

Success codes

Status code	Description
200	Request succeeded
201	Resource created successfully
204	Request succeeded with no response body (common for deletions)

Successful response example

> This is an example only. Response formats may vary by endpoint; refer to the documentation for the specific endpoint.

```
{
  "count": "integer",
  "next": "string|null",
  "previous": "string|null",
  "results": [
    // Returned result content
  ]
}
```

Error codes

Status code	Description
400	Invalid request format
401	Authentication failed

Status code	Description
402	Plan, Credits, or quota insufficient; see 402 responses
403	Insufficient permissions
404	Resource does not exist
409	Conflict with the resource's current state; see 409 responses
429	Rate limit exceeded; see 429 responses
500	Server error

> Error responses primarily take three forms: a detail object (such as {"detail": "…"}, sometimes with a code), a field object (with camelCase field names as keys), or an array of strings. AI Gateway's OpenAI-compatible endpoints use an OpenAI-format error object. Handle errors based on the status code and code, rather than matching the detail text. For full descriptions and handling recommendations, see API Conventions: Error Responses.

Error response examples

400 - Bad request: missing required fields

```
{
  "files": ["This field is required."],
  "filename": ["This field is required."],
  "file": ["This field is required."]
}
```

402 - Insufficient plan, Credits, or quota

`402` covers three cases. Only "no active paid plan" includes a `code`:

Case	`code`
The organization has no active paid plan	`no_active_plan`
The organization has insufficient Credits	None
A member's personal quota is exhausted or unallocated, or their role's quota is exhausted	None

```
{
  "detail": "Your organization has no active paid plan. Please upgrade to
continue using the platform.",
  "code": "no_active_plan"
}
```

```
{
  "detail": "Insufficient credits for KB search. Please top up your account."
}
```

Responses for insufficient Credits and exhausted quotas contain only `detail`, whose text varies by endpoint and language. You cannot distinguish these cases from the response body; ask an organization administrator to check the admin console. AI Gateway endpoints use a different format; see API Conventions: 402.

409 - Conflict with the resource's current state

The request conflicts with the resource's current state, such as when the same job is already running or a resource to be deleted is still referenced by other resources. The response is a `detail` object, and some endpoints include additional fields needed to handle the error:

```
{
  "detail": "An FAQ import is already running for this knowledge base.",
  "jobId": "…"
}
```

Retrieve the resource's latest state before deciding what to do next. Do not resend the request unchanged.

429 - Rate limit exceeded

Exceeding the API key's requests per minute (RPM) or an endpoint's fixed limit returns `429` with a `Retry-After` header (in seconds):

```
{
  "detail": "Request was throttled. Expected available in 42 seconds."
}
```

Wait for the number of seconds specified in `Retry-After` before retrying; do not resend immediately. For the scope of rate limiting, see API Conventions: Rate Limits.

Human Handoff Webhook Events

At each stage of the human handoff workflow—AI determines that a human is needed, assignment to a support agent, return to AI, or queue timeout—MaiAgent sends an event via HTTPS POST to your configured endpoint. This lets dispatching, alerts, CRM, or your own customer service system take over automatically, without polling. Event names describe handoff state transitions and form a fixed public contract. Each event has a corresponding `event_type` in the conversation event log API (`GET /api/v1/conversations/{id}/events/`; see the table below), which you can use for reconciliation or to retrieve the same information through polling.

How to Enable

Customer administrators or integration staff can configure this themselves in the admin console, without engineering assistance:

In the admin console, go to "Customer Service Conversations" → "Conversation Platforms" → Edit the conversation platform to integrate → "Human Handoff Settings" → "Notifications" tab

Enable "Webhook Notifications (System Integration)" and enter the receiver URL (only URLs starting with `https://` are accepted)

(Optional but recommended) Enter a signing secret to enable source verification (see "Signature Verification" below)

Webhook, Notification Center, and Email each have independent switches and do not affect one another. They are disabled by default and enabled per inbox. This channel is separate from the inbox's existing "Message Webhook" for sending and receiving messages.

Event Types

event	Event log API `event_type`	When sent	Notes
`handoff.requested`	`transfer_triggered`	AI determines a human is needed: the conversation enters the queue and AI stops replying automatically	The first event of each handoff workflow; `assignee` is `null`. If human support agents use your own system, this signals them to take over
`handoff.assigned`	`human_assigned`	The conversation is assigned to a	`assignee` identifies the assigned agent; transferring to another

event	Event log API `event_type`	When sent	Notes
		support agent in the MaiAgent admin console (including manual assignment, self-claiming, transfer, and automatic assignment)	agent sends another event with a different event `id`
`handoff.returned_to_ai`	`returned_to_ai`	A support agent clicks "Return to AI Agent" in the admin console or returns the conversation to AI using the `return-to-ai` API	`assignee` is `null` (the conversation no longer has an assigned agent)
`handoff.queue_timeout`	`queue_timeout`	No agent takes over before the inbox's configured "Waiting Timeout" expires, and MaiAgent automatically returns the conversation to AI	`assignee` is `null`. If human support agents use your own system, receiving this means MaiAgent has resumed AI replies without your awareness—treat it as an exception requiring an alert. Alternatively, select "Handover" in dispatch settings and check "No Timeout" (`dispatchMode: "handover"` and `assignmentQueueTimeout: 0` in the API; in automatic dispatch mode, a value of 0 is changed to 300 seconds)

All four event types use exactly the same payload keys, signatures, and retry behavior. The receiver should **route by the `event` field**. New event types may be added in the future; return 2xx and ignore unknown `event` values rather than treating them as errors.

Both `handoff.returned_to_ai` and `handoff.queue_timeout` mean that AI has resumed automatic replies. The former is initiated by a person (an agent or your system), while the latter happens automatically after a timeout. Neither means that the conversation is closed: no event is sent when an agent directly marks a conversation as "Resolved," or when a conversation is requeued because the

agent loses inbox permissions. For complete closure signals, query `status` through the conversation API or observe changes in `conversation.progress_status` in Message Webhooks.

Event Format

```
POST
Content-Type: application/json
```

```
{
  "event": "handoff.assigned",
  "id": "9f0e1d2c-...",
  "occurred_at": "2026-09-09T02:30:00.123456+00:00",
  "conversation": {
    "id": "6c2b7c1e-...",
    "url": "https://admin.maiagent.ai/..."
  },
  "inbox": { "id": "0f9a3d42-..." },
  "organization": { "id": "b1d20c77-..." },
  "contact": { "id": "a4c1f6b3-..." },
  "assignee": { "id": "3e8f5a90-..." }
}
```

Field	Type	Description
event	string	Event name (see “Event Types” above)
id	string (uuid)	Event identifier, identical to the `id` returned by the event log API. Retry payloads are identical; **use this to deduplicate**
occurred_at	string (date-time)	Event occurrence time (ISO 8601). Deliveries are not guaranteed to arrive in order; **use this to sort them**
conversation.id	string (uuid)	Conversation identifier
conversation.url	string (uri)	Link to the conversation page in the admin console
inbox.id	string (uuid)	Inbox identifier
organization.id	string (uuid)	Organization identifier
contact	object	null
assignee	object	null

Field names and nesting match the MaiAgent standard format of the inbox's "Message Webhook" (snake_case, with related objects nested as `{ "id" }`), so the same receiver can handle both Webhook groups. The payload contains only identifiers, the event time, and an admin console link. It **does not include conversation text or the reason for handoff**. To obtain conversation content, use the identifiers to query the conversation and message APIs.

Signature Verification (Recommended)

After you configure a signing secret in the admin console, every delivery includes the following header:

```
X-MaiAgent-Signature-256: sha256=
```

The value is the hexadecimal HMAC-SHA256 digest of the **original request body bytes**, calculated with the secret. Always verify using the original bytes received; do not parse and reserialize them first (differences in field order or whitespace produce a different digest).

```
import hashlib
import hmac


def verify_signature(raw_body: bytes, header_value: str, secret: str) → bool:
    expected = 'sha256=' + hmac.new(secret.encode(), raw_body,
hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, header_value)
```

The secret is write-only: it is not displayed after saving, and the admin console shows only "Configured." Saving with the field empty keeps the existing secret. Enter a new value to replace it, or use "Clear Secret" to remove it. If no secret is configured, deliveries do not include the signature header.

Minimal Receiver Example

The following Python (Flask) receiver can be started directly. It verifies the signature using raw bytes, routes by `event`, and returns 2xx before performing time-consuming work. Set `HANDOFF_WEBHOOK_SECRET` to the signing secret configured in the admin console.

```python
import hashlib
import hmac
import os

from flask import Flask, abort, request

app = Flask(__name__)
SECRET = os.environ["HANDOFF_WEBHOOK_SECRET"]


def verify_signature(raw_body: bytes, header_value: str | None) -> bool:
    if not header_value:
        return False
    expected = "sha256=" + hmac.new(SECRET.encode(), raw_body,
hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, header_value)


@app.post("/webhooks/handoff")
def handoff_webhook():
    raw = request.get_data()  # Original bytes; do not reserialize request.json
    if not verify_signature(raw, request.headers.get("X-MaiAgent-Signature-
256")):
        abort(401)

    event = request.get_json(force=True)
    if already_processed(event["id"]):  # Retries send identical content;
deduplicate by event ID
        return "", 204

    conversation_id = event["conversation"]["id"]
    if event["event"] == "handoff.requested":
        # AI has stopped replying; if agents use your system, create a ticket and
dispatch an agent here
        open_ticket(conversation_id, event["conversation"]["url"])
    elif event["event"] == "handoff.assigned":
        # Assigned to an agent in the MaiAgent admin console; transfer sends
another event with a new assignee
        assign_ticket(conversation_id, event["assignee"]["id"])
    elif event["event"] == "handoff.returned_to_ai":
        close_ticket(conversation_id)
    elif event["event"] == "handoff.queue_timeout":
        # MaiAgent has automatically resumed AI replies; alert if your agents are
still handling the conversation
        close_ticket(conversation_id, reason="queue_timeout")
    # Unknown event: return 2xx and ignore it rather than treating it as an error
```

```
    return "", 204  # Return 2xx within 10 seconds; run time-consuming work in
the background


def already_processed(event_id):
    ...


def open_ticket(conversation_id, url):
    ...


def assign_ticket(conversation_id, assignee_id):
    ...


def close_ticket(conversation_id, reason=None):
    ...
```

```
HANDOFF_WEBHOOK_SECRET=your_secret flask --app receiver run --port 8080
```

For local testing, simulate a delivery with the following commands (calculate the signature using the same secret):

```
BODY='{"assignee":{"id":"3e8f5a90-0000-0000-0000-000000000004"},"contact":
{"id":"a4c1f6b3-0000-0000-0000-000000000005"},"conversation":{"id":"6c2b7c1e-
0000-0000-0000-
000000000001","url":"https://admin.maiagent.ai/conversations/index?
conversationId=6c2b7c1e-0000-0000-0000-
000000000001"},"event":"handoff.assigned","id":"9f0e1d2c-0000-0000-0000-
000000000006","inbox":{"id":"0f9a3d42-0000-0000-0000-
000000000002"},"occurred_at":"2026-09-09T02:30:00.123456+00:00","organization":
{"id":"b1d20c77-0000-0000-0000-000000000003"}}'
SIG=$(printf '%s' "$BODY" | openssl dgst -sha256 -hmac "your_secret" | awk
'{print $NF}')
curl -X POST http://localhost:8080/webhooks/handoff
  -H "Content-Type: application/json"
  -H "X-MaiAgent-Signature-256: sha256=$SIG"
  -d "$BODY"
```

Responses and Retries

The receiver should respond with a 2xx status code within 10 seconds. The response body can contain anything.

On timeout, connection failure, or a non-2xx response, the system retries automatically up to 3 times at 60-second intervals. Retry payloads are identical to the original delivery (with the same `id`); deduplicate using the event `id`. Events are sent as separate background tasks, so `handoff.requested` and the immediately following `handoff.assigned` may arrive out of order. Sort them by `occurred_at`.

Webhook delivery failures do not affect the main handoff workflow or Notification Center and Email notifications.

External System Takeover After Human Handoff

If your human support agents reply in your own customer service system rather than the MaiAgent admin console, combine the events on this page, the conversation platform's "Message Webhook," and the message API for two-way synchronization during handoff. Create a ticket in your system when you receive `handoff.requested`, receive visitor messages through Message Webhooks, send agent replies using `POST /api/v1/messages/outgoing/`, and call `PATCH /api/v1/conversations/{id}/return-to-ai/` when handling is complete to return control to AI. See Third-Party Customer Service System Integration (Including Human Handoff) in the technical manual for the full workflow, request examples, and considerations. Use Scenario A when the visitor's channel is connected to your system, or Scenario B when the visitor uses a MaiAgent conversation platform and only the human support agents use your system.

Integration Workflow

See Scenario B › Integration Workflow in the technical manual.

Request Examples

See Scenario B › Request Examples in the technical manual.

Considerations

See Scenario B › Considerations in the technical manual.

Client Tools

Client tools let AI assistants call tools that run in your application. You declare a list of tools each time you call the conversation API. When the model decides it needs a tool, MaiAgent returns which tool to call and what arguments to pass instead of executing it itself. Your application executes the tool locally and sends the result back to the conversation. The model then uses the result to continue answering or call the next tool.

Use client tools for actions only your application can perform, such as querying systems accessible only on your company's internal network, using a browser extension to interact with the currently open page, or accessing resources on a user's device through an app.

Use Case

An engineer at an e-commerce company integrates an AI assistant into the company's internal customer service console. The order system is accessible only on the internal network. A customer service representative enters “A customer asks when order A1024 will arrive” in the console. When calling the conversation API, the console also declares an order lookup tool. The model returns which order to look up. The console retrieves the shipping status on the internal network and sends it back to the same conversation. The model uses this information to write a reply for the customer. The order system does not need to be exposed externally, and the console sends only the data the model needs.

Requirements

Client tools are available to all organizations without a separate application. For the model to call them, both conditions must be met:

The AI assistant's response mode is set to Agent mode.
The model used by the AI assistant supports tool calling.

> If these conditions are not met, the API does not return an error. It silently ignores the client tools you declare, and the model answers without them. If the model never calls your tools, check the assistant's response mode and model first.

Workflow

Declare Tools and Send a Message

Call `POST /api/v1/chatbots/{chatbotId}/completions/` and include tool definitions in `message.clientTools`.

Receive a Tool Call Request

When the model decides to call your tool, an item with `type` set to `client_tool_call` appears in the response's `contentPayload.items`, and **the turn ends immediately**.

Execute the Tool in Your Application

Execute the corresponding tool using `name` and `input`.

Return the Result to the Same Conversation

Call the conversation API again with the same `conversation`, put the execution result in `message.content`, and **include `clientTools` again**. The model continues answering. If it needs other tools, it returns to step 2 until it produces the final text.

Declare Tools

```
{
  "conversation": "<Conversation ID; optional for the first message>",
  "isStreaming": false,
  "message": {
    "content": "A customer asks when order A1024 will arrive.",
    "clientTools": [
      {
        "name": "get_order",
        "description": "Look up an order's shipping status and estimated delivery
date by order number",
        "inputSchema": {
          "type": "object",
          "properties": {
            "order": { "type": "string", "description": "Order number" }
          },
          "required": ["order"]
        }
      }
    ]
  }
}
```

Field	Type	Description
name	string	Tool name. Only alphanumeric characters, underscores `_`, and hyphens `-` are allowed. Length must be 1–64 characters, and names must be unique within a request.
description	string	A description of the tool's purpose. The model uses it to decide when to use the tool, so write it clearly.
inputSchema	object	JSON Schema for the tool's arguments. Must not exceed 4,000 characters when serialized.

> You can declare up to 20 client tools per request. The tool list is valid only for that request and is not stored in the conversation. Include clientTools again with every subsequent message, including messages that return tool results. Otherwise, the model cannot call these tools in that turn.

Tool Call Request Format

When the model requests a tool call, `contentPayload.items` includes an item like the following. There may be multiple items at once:

```
{
  "type": "client_tool_call",
  "toolCallId": "call_abc123",
  "name": "get_order",
  "input": { "order": "A1024" },
  "startTimestamp": "2026-10-01T08:30:00.000000+00:00"
}
```

Field	Type	Description
type	string	Always `client_tool_call`.
toolCallId	string	null
name	string	Name of the tool to execute, matching the `name` you declared.
input	object	Arguments supplied by the model, corresponding to the `inputSchema` you declared.
startTimestamp	string	null

Non-Streaming and Streaming Differences

Non-streaming (`isStreaming: false`): Tool call items are in `contentPayload.items` in the response body (`done: true`).

Streaming (`isStreaming: true`): Tool call items are sent as separate events. Each event has `done` set to `false` , an empty `content` string, and `client_tool_call` in `contentPayload.items` . The final `done: true` event does not repeat these items, so collect them during streaming.

> All response field names are converted to camelCase, including argument names inside input. For example, order_id defined in your inputSchema becomes orderId in input. Use a single lowercase English word for argument names (such as order or keyword), or accept both forms when parsing.

Return Execution Results

There is currently no dedicated endpoint for submitting tool results. Send the next regular message in **the same conversation**, putting the result in `message.content` :

```
{
  "conversation": "<The same conversation ID>",
  "isStreaming": false,
  "message": {
    "content": "Execution result for tool get_order (toolCallId:
call_abc123):\n<Lookup result>",
    "clientTools": [ "...The same tool definitions as in the previous request..."
]
  }
}
```

Multiple tool calls at once: Execute all of them, then combine each tool's results into a single message.

Include the tool name and `toolCallId` : The backend does not use `toolCallId` for matching, but including it in the result text helps the model identify which call the result belongs to.

Execution failure or user rejection: Explain this in text as well, such as “get_order failed: order not found” or “The user declined this operation.” The model adjusts its answer or uses another approach accordingly.

Result length: Returned content counts toward the model's input. For large amounts of data, extract the required portions first.

Call Multiple Tools Across Turns

Each cycle of “the model requests a call → you return the result” is one turn. The model can call tools over multiple turns until its response no longer contains `client_tool_call` and instead returns the final text. Set a turn limit in your application to prevent an infinite loop if something goes wrong.

Frequently Asked Questions

- The model never calls my tool
- The model says it cannot use tools after I return a result
- I receive a 400 error

Message Webhook

After you configure message webhooks for an Inbox, MaiAgent pushes each message to your system without polling. Common uses include displaying new message notifications in your own system, syncing customer messages to a CRM, and logging AI replies in a data warehouse.

An Inbox can have multiple message webhooks. Each has its own type, URL, and format, and delivers messages independently.

> To receive notifications only for handoffs to human agents, use Handoff Webhook Events. These are sent only when the handoff workflow changes state, rather than for every message. The two settings are independent; you can enable both and handle them with the same receiver.

Types: Incoming and Outgoing

Type	When it is sent	Payload `type`
`Incoming`	Whenever the Inbox receives a customer (contact) message	`incoming`
`Outgoing`	Whenever a reply is sent to the customer's conversation platform, including automatic AI replies and replies from human support agents	`outgoing`

The admin console describes `Incoming` as "Receive external requests," but it actually **pushes each customer message to the URL you configure**.

`Receive` in the same menu works in the opposite direction: an external platform sends messages into MaiAgent. This is outside the scope of this guide. Messages received through `Receive` do not trigger `Incoming` or `Outgoing` webhooks for the same Inbox, preventing messages from looping between the two systems.

To receive both customer messages and replies, configure one `Incoming` webhook and one `Outgoing` webhook. Both can use the same URL; distinguish them by the payload's `type`.

Configuration

Configure in the admin console

Admin console → Customer Conversations → Inboxes → Select an Inbox → Webhook Settings → Add Webhook

Enter a Name, set Type to Incoming or Outgoing, enter the Webhook URL, and select an HTTP Method (default `POST`).

(Recommended) Add an authentication header under Headers, such as `Authorization: Bearer `. The receiver can use it to confirm that requests come from MaiAgent (see Verify the source).

Set Payload Format to MaiAgent Standard (see Payload). Choose Custom Template if you need custom fields.

Make sure Enabled is on, then save. Turning it off pauses delivery and retains the configuration.

The WhatsApp Inbox settings page currently has no Webhook Settings tab. Use the API configuration below instead.

Configure through the API

Message webhooks belong to an Inbox:

Action	Method and path
List	`GET /api/v1/inboxes/{inbox_id}/webhooks/`
Create	`POST /api/v1/inboxes/{inbox_id}/webhooks/`
Retrieve one	`GET /api/v1/inboxes/{inbox_id}/webhooks/{id}`
Update	`PATCH /api/v1/inboxes/{inbox_id}/webhooks/{id}` (send only the fields to change)
Delete	`DELETE /api/v1/inboxes/{inbox_id}/webhooks/{id}`

The API key owner must have permission to manage the Inbox. See API Authentication → API key permissions.

Field	Required	Description
`name`	Yes	A name for easy identification
`direction`	Yes	`incoming` or `outgoing`
`url`	Yes	The URL that receives deliveries; it must be accessible from the internet (`https://` recommended). Delivery to URLs pointing to internal networks or local addresses is blocked
`method`	No	`POST` (default), `PUT`, or `PATCH`
`headers`	No	Custom headers included with every delivery, as a JSON object, such as `{"Authorization": "Bearer ..."}`

Field	Required	Description
`templatePreset`	No	`maiagent_standard` (default), `custom`, or `genesys_cloud` for Genesys Cloud integration
`customTemplate`	Required for `custom`	A Jinja2 template whose rendered output must be JSON. A test render runs when saving; invalid formatting returns 400
`secretToken`	No	The signing secret. When set, each delivery includes an HMAC-SHA256 signature and timestamp headers (see Verify the source). Write-only; never returned in queries
`isEnabled`	No	Whether the webhook is enabled; defaults to `true`

```
curl -X POST "https://api.maiagent.ai/api/v1/inboxes/YOUR_INBOX_ID/webhooks/"
  -H "Authorization: Api-Key YOUR_API_KEY"
  -H "Content-Type: application/json"
  -d '{
    "name": "New message notification",
    "direction": "incoming",
    "url": "https://example.com/webhooks/maiagent-message",
    "secretToken": "YOUR_RANDOM_SECRET"
  }'
```

A successful request returns `201` and the configuration (excluding `secretToken`). To also receive replies, create another webhook with `"direction": "outgoing"`.

Payload

Each message is delivered as a single HTTP request:

Headers: `Content-Type: application/json`, your custom headers, `X-MaiAgent-Delivery-Attempt` (the attempt number, starting at `1`), and, when `secretToken` is set, `X-MaiAgent-Timestamp` and `X-MaiAgent-Signature-256`. Custom headers with the same names as these three headers (case-insensitive) are removed before sending.

Body: JSON generated according to the Payload Format.

MaiAgent Standard format

All fields use snake_case, unlike the camelCase used in API responses. The following is an `Outgoing` (AI reply) payload:

```
{
  "id": "a0c49ae2-013c-4310-b00f-b4554bc9cd83",
  "type": "outgoing",
  "content": "Hello, we have received your order inquiry. Please provide your
order number.",
  "created_at": "2026-10-03T04:59:01.512340+00:00",
  "conversation": {
    "id": "e6b27044-4990-4129-ad8e-ac7649984e7d",
    "title": "Change order address",
    "type": "individual",
    "status": "open",
    "progress_status": "ai_processing",
    "auto_reply_enabled": true,
    "assignee": null
  },
  "inbox": { "id": "6571672b-2fae-4200-a4d0-cf12e86216c3", "name": "Website
support", "channel_type": "web" },
  "contact": { "id": "350691e0-66a5-48ea-855c-d8bb90a7e24a", "name": "Wang
Xiaoming", "email": null, "phone_number": null },
  "sender": { "id": "8a214fc3-75ad-43ba-93f6-567e9df97b76", "name": "Support
assistant", "type": "chatbot" },
  "attachments": [],
  "metadata": {}
}
```


Field	Description
`id`	Message ID; use it for deduplication or look it up through the Message API
`type`	`incoming` (customer message) or `outgoing` (reply); use this field to determine the direction
`content`	Message text
`created_at`	Message creation time (ISO 8601, UTC)
`conversation`	The conversation this message belongs to. A `progress_status` of `human_serving` means a human support agent is handling it. `assignee` is the assigned support agent as `{ "id", "name" }`, or `null` if unassigned
`inbox`	Inbox as `{ "id", "name", "channel_type" }`; examples of `channel_type` include `web`, `line`, and `messenger`
`contact`	The customer (contact); `email` and `phone_number` are `null` when unavailable
`sender`	The speaker. `type` is `contact` (customer), `chatbot` (Agent), or `user` (organization member, such as a human support agent; messages sent on a customer's behalf using an API key

Field	Description
	also have type `user`)
`attachments`	An array of attachments, each containing `id`, `type` (`image` `video` `audio` `sticker` `other`), `filename`, and `source_url`
`metadata`	Additional message information; its contents vary by Inbox

Custom templates

Custom Template (`templatePreset: "custom"` in the API) uses Jinja2 syntax to build the JSON you need. Available variables are `message` (`id`, `type`, `content`, `created_at`, `metadata`), `conversation`, `inbox`, `contact`, `sender`, `sender_type` (`contact` `chatbot` `user`), `attachments`, and `now`. Output strings with the `tojson` filter to prevent quotes or line breaks from breaking the JSON:

```
{
  "text": {{ message.content | tojson }},
  "direction": "{{ message.type }}",
  "conversation_id": "{{ conversation.id }}",
  "customer": {{ (contact.name if contact else none) | tojson }}
}
```

If the template fails to render at delivery time, that message is not sent.

Verify the source

When you set `secretToken` (the API field, or Secret Token in the admin console's webhook form; this field is hidden in the console when the payload format is Genesys Cloud), each delivery includes these headers:

Header	Content
`X-MaiAgent-Timestamp`	Unix time at the moment of sending, in seconds
`X-MaiAgent-Signature-256`	`sha256=` followed by the hexadecimal value of `HMAC-SHA256(secretToken, "{timestamp}." + raw body)`

Verification steps for the receiver:

Calculate the signature using the **original raw body bytes** you received. Do not parse the JSON and serialize it again first; the bytes will differ and the signature will not match.

Compare it against `X-MaiAgent-Signature-256` using a constant-time comparison, such as Python's `hmac.compare_digest`.

Reject requests when `X-MaiAgent-Timestamp` differs from the receiver's current time by more than the allowed tolerance (for example, 5 minutes) to prevent replay attacks. The timestamp and signature are recalculated at send time for every attempt, including retries.

For details, see Verify signatures and timestamps in the technical manual.

If you do not use signatures, you can set an authentication header in `headers`, such as `Authorization: Bearer`. Values in `headers` are returned unchanged when querying webhook settings and are visible to any organization member who can view that Inbox's settings. `secretToken` is write-only and is never returned.

When `secretToken` is set, the legacy `X-Webhook-Secret` header (containing the raw `secretToken` value) is also sent for compatibility with existing receivers. New integrations should verify the signature instead.

Use a sufficiently long random secret for authentication and receive requests over HTTPS.

Responses and retries

Delivery runs in the background after the message is saved, usually arriving within a few seconds. Messages are sent independently, and **arrival order is not guaranteed**. Sort by `created_at` and deduplicate by `id`.

The receiver must return a 2xx response within **30 seconds**. Timeouts, connection failures, and non-2xx responses count as failures.

Temporary failures are retried automatically: timeouts, connection failures, `5xx`, and `429` are retried once each after approximately 1, 5, and 15 minutes, for a maximum of 3 retries (4 attempts including the initial delivery). Other `4xx` responses, destinations blocked by security protections, and custom template rendering failures are not retried. The retry body is identical to the initial body; `X-MaiAgent-Delivery-Attempt` increments to `2`, `3`, and `4`. Intervals are fixed; the response's `Retry-After` is not used. Retries stop if the webhook is disabled or deleted, or the message is deleted, before the next attempt. See Automatic retries in the technical manual for details.

With retries, a message can be delivered “at least once”: if the receiver has processed it but its response times out or returns `5xx`, the same message will arrive again. Deduplicate by message `id`. Save the content before responding with 2xx and move time-consuming processing to the background. Return `4xx` only when the message can never be processed; return `5xx` or `429` for temporary problems, such as a downstream service outage.

Delivery stops after all 3 retries fail; message webhooks **do not guarantee delivery**. The polling approach in Detect new messages can check for missed **customer messages** (Incoming) at a low frequency. AI and support replies (Outgoing) often appear after the customer message, and the polling

example may not read them again, so it cannot recover replies whose webhook delivery failed. To identify failed deliveries, query the delivery records described below. A `statusCode` that is not 2xx or is empty (timeout or connection failure) indicates a failed delivery. `requestBody` contains the payload sent at the time (the MaiAgent Standard format includes the message `id` and `conversation.id`); then read the first page of that conversation's message list to recover the missing messages.

Each attempt, including retries, creates its own delivery record with the URL, body, and response status code. Query them with `GET /api/v1/chatbots/{chatbot_id}/webhook-records/` (requires edit permission for that Agent; `chatbot_id` is the Agent linked to the Inbox). This API also returns camelCase fields, while the actual delivered payload uses snake_case.

Failed webhook deliveries do not affect message sending, receiving, or AI replies.

Minimal receiver example

The following Python (Flask) receiver can be started directly. It verifies signatures and timestamps, deduplicates by message `id` (the same message may arrive again on retries), routes by `type`, and returns 2xx before processing. Set `MESSAGE_WEBHOOK_SECRET` to your configured `secretToken`. If you use a custom authentication header instead, read the corresponding header.

```python
import hashlib
import hmac
import os
import time

from flask import Flask, abort, request

app = Flask(__name__)
SECRET = os.environ["MESSAGE_WEBHOOK_SECRET"]
TOLERANCE_SECONDS = 300
processed_ids: set[str] = set()  # In production, use a database or Redis and set a retention period


def is_valid_signature(timestamp: str, signature: str, body: bytes) -> bool:
    if not timestamp.isdigit() or abs(time.time() - int(timestamp)) > TOLERANCE_SECONDS:
        return False
    expected = "sha256=" + hmac.new(SECRET.encode(), f"{timestamp}.".encode() + body, hashlib.sha256).hexdigest()
    return hmac.compare_digest(signature, expected)


@app.post("/webhooks/maiagent-message")
def message_webhook():
    if not is_valid_signature(
        request.headers.get("X-MaiAgent-Timestamp", ""),
        request.headers.get("X-MaiAgent-Signature-256", ""),
        request.get_data(),
    ):
        abort(401)

    message = request.get_json(force=True)
    if message["id"] in processed_ids:
        return "", 204
    processed_ids.add(message["id"])

    conversation_id = message["conversation"]["id"]
    if message["type"] == "incoming":
        notify_new_message(conversation_id, message["content"])  # For example, display a new message notification
    elif message["type"] == "outgoing":
        archive_reply(conversation_id, message["content"])

    return "", 204  # Return 2xx within 30 seconds; timeouts or 5xx trigger retries. Run slow tasks in the background
```

```
def notify_new_message(conversation_id, content):
    ...


def archive_reply(conversation_id, content):
    ...
```

```
MESSAGE_WEBHOOK_SECRET=YOUR_SECRET flask --app receiver run --port 8080
```

Use the following commands to simulate a delivery with a valid signature for local testing:

```
BODY='{"id":"acadbd9c-0000-0000-0000-000000000001","type":"incoming","content":"I
want to change my order address","created_at":"2026-10-
03T04:58:55.262828+00:00","conversation":{"id":"e6b27044-0000-0000-0000-
000000000002","title":"Change order
address","type":"individual","status":"open","progress_status":"ai_processing","a
uto_reply_enabled":true,"assignee":null},"inbox":{"id":"6571672b-0000-0000-0000-
000000000003","name":"Website support","channel_type":"web"},"contact":
{"id":"350691e0-0000-0000-0000-000000000004","name":"Wang
Xiaoming","email":null,"phone_number":null},"sender":{"id":"350691e0-0000-0000-
0000-000000000004","name":"Wang Xiaoming","type":"contact"},"attachments":
[],"metadata":{}}'
TS=$(date +%s)
SIG=$(printf '%s.%s' "$TS" "$BODY" | openssl dgst -sha256 -hmac "YOUR_SECRET" |
sed 's/^.* //')
curl -X POST http://localhost:8080/webhooks/maiagent-message
  -H "Content-Type: application/json"
  -H "X-MaiAgent-Timestamp: $TS"
  -H "X-MaiAgent-Signature-256: sha256=$SIG"
  --data-raw "$BODY"
```

Detect New Messages

To display a "new message" notification in your system or notify human support agents when a customer sends a message, choose an approach in this order:

Use Webhook delivery first: The conversation platform's message Webhook sends one notification per message, with no polling required (see Use Webhooks Instead of Polling).

If you only need human handoff notifications: Use human handoff Webhook events, which are sent only when the handoff workflow changes state.

If polling is required: Follow the recommended polling approach to find only updated conversations and read only the first page of each conversation's message list.

> Do not fetch the entire conversation history from newest to oldest in every polling round. A conversation with hundreds of messages requires reading several pages per round, and each page is a full query. Doing this for multiple conversations every few seconds slows down both your polling and the platform's overall response time.

Use Webhooks Instead of Polling

Requirement	Recommended approach	When notifications are sent
Receive a notification for every customer message	Use the conversation platform's message Webhook with type `Incoming`	For every customer (contact) message
Synchronize replies sent by AI or support agents	Use the conversation platform's message Webhook with type `Outgoing`	For every reply sent to the customer's conversation platform
Receive a notification when a conversation needs a human agent	Human handoff Webhook events	Only for human handoff, agent assignment, return to AI, or queue timeout; **not** for every message

Configuration path: Admin console → Customer Conversations → Conversation Platforms → select a conversation platform → Webhook Settings → Add Webhook. Set Type to Incoming or Outgoing (although the admin console describes Incoming as "Receive external requests," it actually sends each customer message to your configured URL; Receive is the type that receives external messages). Set Payload Format to MaiAgent Standard (MaiAgent's standard format). See Message Webhook for payload contents, source verification, and retry behavior.

The WhatsApp conversation platform's settings page currently has no Webhook Settings tab. Use the polling approach below instead.

The two Webhook types are configured independently and can be enabled together; deduplicate their payloads using the message or event `id`. Message Webhooks automatically retry up to 3 times on timeouts, connection failures, `5xx`, or `429` responses (the same message may arrive more than once). If all 3 retries fail, delivery stops; delivery is not guaranteed. The polling approach below can check for missed **customer messages** (Incoming) at a low frequency. If delivery of an AI or support agent reply (Outgoing) fails, the polling example may not retrieve it. Check the Webhook delivery logs for failed deliveries instead (see Message Webhook → Responses and Retries).

Recommended Polling Approach

If you cannot receive Webhooks (for example, because the receiving endpoint cannot be exposed publicly), poll using the following two APIs together.

1. Find Updated Conversations

```
GET /api/v1/conversations/?inbox=&page_size=20
```

The conversation list is always sorted **by last message time, newest first** (`lastMessageCreatedAt` in the response). For each conversation, record the last observed `lastMessageCreatedAt` and the last message's `lastMessage.id`. A change to either means the conversation has been updated (timestamps have second precision, so if two messages arrive in the same second, only the `id` changes). In each polling round, read from the first entry down and stop when you reach a conversation older than "the latest time observed so far minus 10 minutes." Usually, you only need the first page. Follow the response's `next` to the next page only if more than `page_size` conversations were updated within that window.

The extra 10-minute lookback is necessary because, in API conversations, a conversation's `lastMessageCreatedAt` may be written only after the AI reply finishes generating. The value is still the customer message's timestamp, so it may be older than other conversations you saw in the previous round. Comparing only whether it is newer than the previous round's latest time would miss these conversations.

2. Read Only the First Page of Messages for Each Updated Conversation

```
GET /api/v1/messages/?conversation=&page_size=20
```

The **first page of the message list contains the latest messages**: the system selects the latest `page_size` messages, then sorts them from oldest to newest within the page. The last message is

therefore the latest in the entire conversation. The response's `next` points to **older** messages; do not read back through the history in every round when detecting new messages.

Use each message's `id` to determine whether it has been processed: unprocessed `id` values on the first page identify new messages. Pagination is needed in only one case—when **all** messages on the first page are unprocessed (more than `page_size` messages arrived between rounds). Only then follow `next` to older pages until you encounter a processed message.

3. Polling Interval and Page Size

Setting	Recommendation	Description
Polling interval	30–60 seconds	Use Webhooks for more immediate notifications; do not shorten the interval to a few seconds
`page_size`	20	Default 20, maximum 100. Larger pages produce slower responses and more content

- Response fields use camelCase. lastMessageCreatedAt and createdAt are timestamp strings in milliseconds (for example, "1790753081000"), but have only second precision. Multiple messages may arrive within the same second; deduplicate by message id, rather than comparing timestamps alone. - The example uses the local time as the starting point for detection. Enable network time synchronization (NTP) on the host to prevent a fast clock from skipping messages at startup. - The conversation list includes only conversations with customer messages. Conversations containing only an opening greeting, where the customer has not yet spoken, do not appear. - lastMessageCreatedAt always updates when a new customer message arrives, but the update may occur after the message itself is written (see step 1 above). An automatic AI reply does not always update it. This polling approach therefore detects only customer messages: an AI reply is retrieved along with them only if it has already been generated at the time of reading. If an AI reply is generated after the customer message has been read and does not change the conversation's last message time or last message, subsequent polling skips that conversation and will not retrieve the reply. If generating a single AI reply takes more than 10 minutes (for example, due to a long-running tool), detection of the customer message may also be delayed until the conversation is updated again. To receive every customer message and reply promptly, use Incoming Outgoing message Webhooks (temporary failures are retried up to 3 times, but delivery is still not guaranteed).

Request Examples

cURL

```
# 1. List the conversation platform's conversations by last message time, newest first
curl "https://api.maiagent.ai/api/v1/conversations/?inbox=YOUR_INBOX_ID&page_size=20"
  -H "Authorization: Api-Key YOUR_API_KEY"

# 2. Read only the first page of messages for updated conversations (latest 20, oldest first within the page)
curl "https://api.maiagent.ai/api/v1/messages/?conversation=CONVERSATION_ID&page_size=20"
  -H "Authorization: Api-Key YOUR_API_KEY"
```

Python

```python
import os
import time

import requests

API = "https://api.maiagent.ai/api/v1"
HEADERS = {"Authorization": f"Api-Key {os.environ['MAIAGENT_API_KEY']}"}
INBOX_ID = os.environ["INBOX_ID"]
POLL_SECONDS = 30
PAGE_SIZE = 20
LOOKBACK_MS = 10 * 60 * 1000  # Look back an extra 10 minutes: the conversation's last message time may be written only after the AI reply is generated


def get(url: str, params: dict | None = None) -> dict:
    resp = requests.get(url, headers=HEADERS, params=params, timeout=10)
    resp.raise_for_status()
    return resp.json()


def recent_conversations(since_ms: int) -> list[dict]:
    """Conversations are sorted by last message time, newest first: stop at entries older than since_ms; usually only the first page is needed."""
    conversations = []
    url, params = f"{API}/conversations/", {"inbox": INBOX_ID, "page_size": PAGE_SIZE}
    while url:
        page = get(url, params)
        for conv in page["results"]:
            if int(conv["lastMessageCreatedAt"]) < since_ms:
                return conversations
            conversations.append(conv)
        url, params = page["next"], None
    return conversations


def new_messages(conversation_id: str, seen_ids: set[str], not_before_ms: int) -> list[dict]:
    """Read the first page (latest messages, oldest first within the page); read older pages only if the entire page is unprocessed."""
    found = []
    url, params = f"{API}/messages/", {"conversation": conversation_id, "page_size": PAGE_SIZE}
    while url:
        page = get(url, params)
        fresh = [m for m in page["results"] if m["id"] not in seen_ids and int(m["createdAt"]) >= not_before_ms]
```

```
        found = fresh + found
        if len(fresh) < len(page["results"]):
            break  # This page contains a processed message (or one from before
detection started); no need to read further back
        url, params = page["next"], None
    return found


def change_key(conv: dict) -> tuple:
    """Timestamps have second precision: for two messages in the same second,
only the last message id changes, so compare both."""
    return conv["lastMessageCreatedAt"], (conv.get("lastMessage") or
{}).get("id")


def main() -> None:
    started_ms = int(time.time()) * 1000  # Start detecting now (rounded down to
the second), without processing historical messages
    newest_ms = started_ms
    last_seen: dict[str, tuple] = {}  # Last observed change_key for each
conversation (persist or periodically clean up in production)
    seen_ids: dict[str, set[str]] = {}  # Processed message ids for each
conversation (same as above)
    while True:
        try:
            round_newest = newest_ms
            for conv in recent_conversations(newest_ms - LOOKBACK_MS):
                if last_seen.get(conv["id"]) == change_key(conv):
                    continue  # No new customer messages in this conversation (AI
replies generated later will not be read here)
                ids = seen_ids.setdefault(conv["id"], set())
                for msg in new_messages(conv["id"], ids, started_ms):
                    print(f"[New message] Conversation {conv['id']}
{msg['type']}: {msg['content'][:40]}")
                    ids.add(msg["id"])
                last_seen[conv["id"]] = change_key(conv)
                round_newest = max(round_newest,
int(conv["lastMessageCreatedAt"]))
            newest_ms = round_newest  # Advance only after the entire polling
round succeeds
        except requests.RequestException as exc:
            print(f"Query failed this round; retrying next round: {exc}")  # The
lookback starting point has not advanced; unfinished conversations will be picked
up next round
        time.sleep(POLL_SECONDS)
```

```
if __name__ == "__main__":
    main()
```

An API key has the same permissions as its owner’ s role in the organization. The owner must be able to view conversations on the relevant conversation platform. See API Authentication → API Key Permission Scope.

Parser Auto-Detection Results

When you upload a file to a knowledge base without specifying `parser` (leaving the parser set to "Auto-detect (default)"), the system selects a parser based on the file's content. In the responses from List files in a knowledge base and Retrieve file details, each file's `parserAutoDetection` reports which parser was selected, why, and the metrics calculated during detection. Use it in your own system to distinguish automatically selected parsers from user-selected parsers and display the reason for the selection.

Fields

Field	Type	Description
`parser`	object	The parser selected by auto-detection and currently used for this file. Its fields match `parser` in the file response (`id`, `name`, `provider`, `isTimestampSttProvider`)
`reason`	string	Reason category; see Reason categories
`facts`	object	Metrics calculated during detection; see Detection metrics. An empty object `{}` when no metrics are available
`detectedAt`	string	Detection time as a Unix timestamp in milliseconds, represented as a string (see Field format)

The metrics in `facts` are calculated at detection time and are not recalculated afterward. Reparsing with the same parser does not trigger detection again, so the original metrics and detection time are retained.

When the result is null

A `null` value for `parserAutoDetection` means that the file's current parser was not selected by auto-detection, or no detection result is available yet:

A parser was explicitly specified during upload or reparsing (selected by the user).

"Batch reparse files" was used to replace the automatically selected parser with another parser. Reparsing with the same parser retains the original result.

"Batch reparse files" was used to switch back to auto-detection (the file's entry explicitly includes `"parser": null`), and the new detection is still queued or incomplete. Once detection finishes, the new result replaces it.

The file was uploaded before this feature was introduced (existing files are not backfilled).

When switching back to auto-detection through Batch reparse files, explicitly include "parser": null in the file's entry. If the entry has no parser key, the file's parser is not changed, the file is not reparsed, and parserAutoDetection remains unchanged. During upload, you can simply omit parser.

Response examples

The PDF has many scanned pages, so the system selects the Vision parser:


```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "filename": "合約掃描檔.pdf",
  "parser": {
    "id": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
    "name": "Vision Parser",
    "provider": "anthropic",
    "isTimestampSttProvider": false
  },
  "parserAutoDetection": {
    "parser": {
      "id": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
      "name": "Vision Parser",
      "provider": "anthropic",
      "isTimestampSttProvider": false
    },
    "reason": "scanned",
    "facts": {
      "analyzedPages": 50,
      "partial": true,
      "nonBlankPages": 48,
      "scannedPages": 40,
      "imagePages": 42
    },
    "detectedAt": "1760860800000"
  }
}
```


The user selected a parser during upload:

```
{
  "id": "550e8400-e29b-41d4-a716-446655440001",
  "filename": "產品手冊.docx",
  "parserAutoDetection": null
}
```

These examples show only the relevant fields. See each endpoint's response schema for other file fields.

Reason categories

`reason`	Meaning	`facts`
`scanned`	The content is mostly scanned pages (PDF or Word), requiring image interpretation	PDF or Word metrics
`image_pages`	The PDF has enough pages requiring image interpretation (scanned pages or other pages dominated by images)	PDF metrics
`large_images`	The Word document contains enough large images in its body	Word metrics
`unmeasurable_images`	The Word document contains images whose dimensions cannot be measured, so it is treated as requiring image interpretation	Word metrics, possibly an empty object
`presentation`	A presentation file (PPTX)	Empty object
`no_image_content`	No pages require image interpretation and there are no large images in the body	PDF or Word metrics; an empty object for Word files with no images at all
`below_threshold`	Some pages require image interpretation or the body contains large images, but the threshold for selecting the Vision parser is not met	PDF or Word metrics
`no_content`	All PDF pages analyzed are blank	PDF metrics
`file_type_not_detected`	Content detection is not performed for this file type (such as legacy PPT presentations); the system uses the default parser for the type	Empty object
`detection_failed`	An error occurred during detection; the system uses the default parser for the file type	Empty object

`reason`	Meaning	`facts`
`vision_unavailable`	The content requires image interpretation, but no Vision parser is available; `parser` is the parser actually used	Same as the content category (PDF or Word metrics; an empty object for PPTX)

Detection rules may introduce additional reason categories in the future. Avoid hard-coding logic based only on this table. For an unrecognized reason, show a generic description (such as “The system automatically selected a parser based on the file's content”) instead of raising an error or skipping the entire file record.

Detection metrics

The keys in `facts` vary by file type. As shown above, `facts` is an empty object `{}` for PPTX, Word files with no images at all, file types that do not undergo content detection, and detection failures.

PDF

Key	Type	Description
`analyzedPages`	integer	Number of pages examined during detection
`partial`	boolean	Whether only the first pages were examined (`true` when the file exceeds the detection limit, currently the first 50 pages)
`nonBlankPages`	integer	Number of nonblank pages among the examined pages
`scannedPages`	integer	Number of scanned pages among the examined pages
`imagePages`	integer	Number of examined pages requiring image interpretation (including scanned pages)

Word (DOCX)

Key	Type	Description
`figures`	integer	Number of large images in the body
`textChars`	integer	Number of characters in the body

For Word files with no images at all (`no_image_content`) or whose image dimensions cannot be measured (`unmeasurable_images`), `facts` may be an empty object. Treat each key as optional when

reading it.